UFMT

UNIVERSIDADE FEDERAL DE MATO GROSSO
INSTITUTO DE COMPUTAÇÃO
COORDENAÇÃO DE ENSINO DE GRADUAÇÃO EM
CIÊNCIA DA COMPUTAÇÃO

UMA HEURÍSTICA BRKGA PARA O PROBLEMA DA COBERTURA DE CONJUNTOS COM CONFLITOS ENTRE CONJUNTOS.

ANDRÉ ARCE VERA ÁVILA

CUIABÁ - MT

2026

UFMT

UNIVERSIDADE FEDERAL DE MATO GROSSO
INSTITUTO DE COMPUTAÇÃO
COORDENAÇÃO DE ENSINO DE GRADUAÇÃO EM
CIÊNCIA DA COMPUTAÇÃO

UMA HEURÍSTICA BRKGA PARA O PROBLEMA DA COBERTURA DE CONJUNTOS COM CONFLITOS ENTRE CONJUNTOS.

ANDRÉ ARCE VERA ÁVILA

Orientador: Prof. Dr. Eduardo Theodoro Bogue

Monografia apresentada ao Curso de Ciência da Computação, do Instituto de Computação da Universidade Federal de Mato Grosso, como requisito para obtenção do título de Bacharel em Ciência da Computação

CUIABÁ - MT

2026

RESUMO

Neste trabalho, exploramos uma variante do Set Covering Problem, a qual apresenta conflitos entre subconjuntos. Caso dois subconjuntos em conflito sejam selecionados para uma solução, uma determinada penalidade será paga.

Um algoritmo que utiliza uma heurística do tipo Biased Random-Key Genetic Algorithm (BKGA), que funciona de maneira iterativa, aplicando diversas técnicas baseadas em conceitos da biologia, será desenvolvido com o objetivo de entregar soluções de boa qualidade para o problema, ou seja, soluções de baixo custo associado. A variante do BRKGA adotada é o modelo ilha, que trabalha com múltiplas populações que, a princípio, evoluem de maneira isolada e posteriormente o algoritmo realiza trocas de indivíduos entre elas a fim de promover diversidade genética.

Palavras-chaves: Otimização Combinatória, Set Covering Problem, Conflitos, Algoritmos Genéticos, Heurística, Biased Random-Key Algorithm.

ABSTRACT

This work explores a variant of the Set Covering Problem that introduces conflicts among subsets. If two conflicting subsets are selected in a solution, a specific penalty cost is incurred.

To address this, we develop an algorithm based on the Biased Random-Key Genetic Algorithm (BRKGA) metaheuristic. This iterative approach applies biologically inspired mechanisms to deliver high-quality, low-cost solutions to the problem. The adopted BRKGA variant relies on a multi-island model architecture, which maintains multiple populations that initially evolve in isolation. Subsequently, the algorithm performs a migration process, exchanging individuals across populations to promote genetic diversity.

Key-words: Combinatorial Optimization, Set Covering Problem, Conflicts, Genetic Algorithms, Heuristics, Biased Random-Key Algorithm.

SUMÁRIO

1 INTRODUÇÃO **1**

2 REVISÃO BIBLIOGRÁFICA **5**

3 HEURÍSTICA BRKGA **7**
3.1 Pré-processamento: Pesos de Escassez **8**
3.2 Decodificador Híbrido **9**
3.2.1 Métrica de Seleção 10
3.2.2 Algoritmo do Decodificador 10
3.3 Dinâmica Evolutiva Específica **12**
3.4 Modelo de Ilhas (BRKGA-Island-Model) **13**

4 RESULTADOS COMPUTACIONAIS **15**
4.1 Caracterização das Instâncias e Ambiente de Teste **15**
4.2 Parametrização do Algoritmo **16**

5 CONCLUSÃO **20**

A RESULTADOS DETALHADOS POR INSTÂNCIA **22**

REFERÊNCIAS **26**

LISTA DE ILUSTRAÇÕES

Figura 1 – Exemplo de 2 soluções para o Problema da Cobertura de Conjuntos. O custo ótimo para esta instância é 17. 2
Figura 2 – Transição da população k para a população $k+1$. 12
Figura 3 – Estratégia de cruzamento do BRKGA. 12

LISTA DE TABELAS

Tabela 1 – Caracterização das instâncias de teste utilizadas (*Benchmark* SCP-CS). 16
Tabela 2 – Parâmetros configurados para os experimentos computacionais. 17
Tabela 3 – Resultados médios agrupados por conjunto e Média Geral. 18
Tabela 4 – Comparação de resultados: Parallel GRASP vs. BRKGA-Island (Proposto). 23

LISTA DE ABREVIATURAS E SIGLAS

PLI	Programação Linear Inteira
BRKGA	Biased Random-Key Genetic Algorithm
GA	Genetic Algorithm
GRASP	Greedy Randomized Adaptive Search Procedure
SCP	Set Covering Problem
SCP-CS	Set Covering Problem with Conflicts on Sets

CAPÍTULO 1

INTRODUÇÃO

Problemas de otimização combinatória estão presentes em diversas aplicações reais, como logística, telecomunicações, alocação de recursos e planejamento. Um dos problemas mais conhecidos dessa área é o Problema da Cobertura de Conjuntos (em inglês, *Set Covering Problem* – SCP), cuja formulação consiste em: dado um conjunto universo de elementos e uma coleção de subconjuntos (que contém elementos do conjunto universo) com custos associados, selecionar um conjunto de subconjuntos que cubra todos os elementos ao menor custo possível, que será a solução. Esse problema é conhecido por ser NP-difícil, e sua complexidade cresce rapidamente à medida que o número de elementos e subconjuntos disponíveis aumenta. Muitas aplicações práticas impõem restrições adicionais que tornam o problema ainda mais desafiador. Uma dessas variações é o Problema da Cobertura de Conjuntos com Conflitos entre Subconjuntos (SCP-CS), variação na qual é considerada uma penalidade para subconjuntos em conflito.

O Problema da Cobertura de Conjuntos pode ser formalmente definido conforme apresentado em (CAPRARA; TOTH; FISCHETTI, 2000). Seja $A = (a_{ij})$ uma matriz binária de dimensão $m \times n$, e $c = (c_j)$ um vetor de custos de dimensão n. Definem-se os conjuntos $M = \{1, \ldots, m\}$ e $N = \{1, \ldots, n\}$, correspondendo, respectivamente, às linhas e colunas da matriz A. Cada valor c_j, com $j \in N$, representa o custo associado à coluna j. Diz-se que a coluna $j \in N$ cobre a linha $i \in M$ se $a_{ij} = 1$. O objetivo do SCP é determinar

um subconjunto $S \subseteq N$ de colunas, de custo total mínimo, de tal forma que cada linha $i \in M$ seja coberta por ao menos uma coluna $j \in S$.

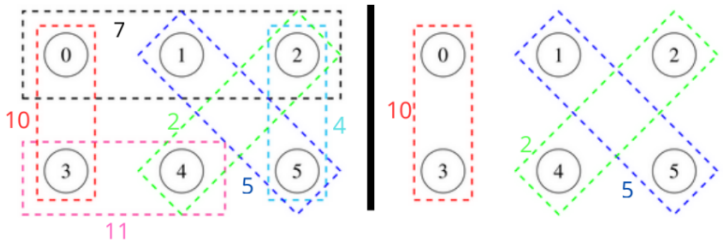

Figura 1 – Exemplo de 2 soluções para o Problema da Cobertura de Conjuntos. O custo ótimo para esta instância é 17.

Uma instância do SCP é composta pelo conjunto universo (que contém os elementos a serem cobertos) e um conjunto de subconjuntos compostos por permutações dos elementos do conjunto universo. Na Figura 1, apresenta-se um exemplo ilustrativo do SCP, onde cada subconjunto e seu respectivo custo são apresentados na mesma cor. São apresentadas duas soluções, na solução mais à esquerda da figura, todos os elementos (representados pelos nós numerados de 1 a 5) estão cobertos por, pelo menos, um subconjunto, o que caracteriza uma solução viável, cujo custo total é 39, valor dado pela soma dos custos de todos os subconjuntos selecionados para a solução. Observa-se, entretanto, que alguns elementos, como os nós 2 e 4, estão sendo cobertos por múltiplos subconjuntos simultaneamente.

Considerando que cada subconjunto possui um custo associado, a presença de coberturas redundantes aumenta, desnecessariamente, o custo total da solução. Assim, é desejável minimizar essas redundâncias, buscando uma cobertura mais eficiente. No lado direito da figura, observa-se uma cobertura igualmente completa dos primeiros cinco nós, porém utilizando apenas 3 dos 6 subconjuntos empregados na solução à esquerda. Somando os custos de todos os subconjuntos utilizados na solução à direita, obtemos o valor 17. Essa segunda solução apresenta um custo total menor, sendo, portanto, uma solução válida (pois cobre todos os elementos) e mais eficiente para o problema.

O SCP tradicional não leva em consideração as relações entre os subconjuntos selecionados, o que pode resultar em soluções nas quais os subconjuntos compartilham muitos elementos. Em diversos contextos práticos, como alocação de recursos com incompatibilidades ou restrições operacionais, esse comportamento pode ser indesejável ou mesmo inviável. Uma extensão importante, o Problema da Cobertura de Conjuntos com Conflitos entre Conjuntos (em inglês, Set Covering Problem With Conflicts on Subsets - SCPCS), aborda essa limitação ao considerar conflitos entre pares de subconjuntos. O

modelo, apresentado em (CARRABS et al., 2024), define uma custo de penalidade se dois subconjuntos em conflito forem selecionados para a mesma solução. Além dos custos de seleção, introduz-se uma penalidade $d_{jl} \in \mathbb{R}_0^+$ para a escolha simultânea dos subconjuntos U_j e U_l. O conjunto D representa todos os pares $\{j,l\}$ em conflito, ou seja, aqueles para os quais $d_{jl} > 0$. O objetivo do SCP-CS é encontrar uma cobertura que minimize a soma dos custos de cobertura e das penalidades de conflito.

Para o SCP-CS, a formulação de Programação Linear Inteira (PLI) proposta em (CARRABS et al., 2024) baseia-se em dois conjuntos de variáveis de decisão. A variável binária x_j indica a seleção do subconjunto U_j, assumindo valor 1 se selecionado e 0 caso contrário. A variável auxiliar y_{jl} captura a ocorrência de conflitos, assumindo valor 1 se, e somente se, o par de subconjuntos conflitantes $\{j,l\} \in D$ for selecionado simultaneamente. O modelo é formulado da seguinte maneira:

$$
\begin{aligned}
\textbf{(SCP-CS_BLP)} \quad \min \quad & \sum_{j \in N} c_j x_j + \sum_{\{j,l\} \in D} d_{jl} y_{jl} & & (1)\\
\text{sujeito a} \quad & \sum_{j \in N} a_{ij} x_j \geq 1, & \forall i \in U \quad & (2)\\
& x_j + x_l \leq y_{jl} + 1, & \forall \{j,l\} \in D \quad & (3)\\
& x_j \in \{0,1\}, & \forall j \in N \quad & (4)\\
& y_{jl} \in \{0,1\}, & \forall \{j,l\} \in D \quad & (5)
\end{aligned}
$$

A Função Objetivo (1) minimiza o custo total da solução, composto pela soma dos custos dos subconjuntos selecionados e das penalidades decorrentes dos conflitos ativados. A Restrição (2) garante a viabilidade da solução, assegurando que cada elemento $i \in U$ seja coberto por pelo menos um subconjunto (onde $a_{ij} = 1$ se $i \in U_j$, e 0 caso contrário). A Restrição (3) modela a lógica do conflito: se ambos os subconjuntos x_j e x_l forem selecionados (soma igual a 2), a variável y_{jl} é forçada a assumir o valor 1 para satisfazer a desigualdade, ativando a penalidade na função objetivo. Por fim, (4) e (5) definem o domínio binário das variáveis.

Os parâmetros de penalidade d_{jl} são calculados conforme a metodologia de (CARRABS et al., 2024). Define-se primeiro um fator de escala γ, que representa o custo unitário máximo por elemento, normalizado pela cardinalidade dos subconjuntos:

$$\gamma = \max \left\{ \left\lceil \max_{j \in N} \frac{c_j}{|U_j|} \right\rceil, 1 \right\}$$

Com base em γ, a penalidade d_{jl} para um par $\{j,l\}$ é proporcional ao número de elementos na interseção que excede um limite de tolerância k:

$$d_{jl} = \gamma \max\{|U_j \cap U_l| - k, 0\}, \quad \forall j, l \in N, j \neq l$$

Dessa forma, se a interseção $|U_j \cap U_l|$ for menor ou igual a k, não há penalidade ($d_{jl} = 0$). Caso contrário, aplica-se o custo γ para cada elemento excedente.

Para ilustrar a relevância prática do modelo, considere o planejamento de uma rede de telecomunicações. O objetivo é instalar torres para cobrir uma área geográfica (conjunto U) com custo mínimo. Entretanto, a instalação de torres muito próximas pode gerar interferência eletromagnética (conflito). O modelo SCP-CS permite selecionar os locais de instalação, equilibrando o custo de construção e a penalidade por interferência, evitando áreas de cobertura excessivamente sobrepostas que degradam o sinal.

Para a resolução eficiente deste problema, este trabalho propõe uma metaheurística baseada no *Biased Random-Key Genetic Algorithm* (BRKGA). Algoritmos Genéticos são métodos iterativos inspirados na evolução biológica, onde uma população de soluções candidatas (cromossomos) evolui através de seleção, cruzamento e mutação. O BRKGA, especificamente, diferencia-se por utilizar vetores de chaves aleatórias (números reais) para representar as soluções, empregando um decodificador para mapear o genótipo em fenótipo e um mecanismo de cruzamento enviesado que favorece a herança de genes de indivíduos com melhor aptidão (*fitness*).

O restante deste trabalho está organizado da seguinte forma: o Capítulo 2 apresenta a revisão bibliográfica e trabalhos relacionados. A metodologia proposta e a implementação do BRKGA são descritas no Capítulo 3. O Capítulo 4 discute os experimentos computacionais realizados. Por fim, o Capítulo 5 conclui o trabalho e traz ideias para continuidade da pesquisa.

CAPÍTULO 2

REVISÃO BIBLIOGRÁFICA

O Problema de Cobertura de Conjuntos (SCP) foi formalmente caracterizado no trabalho de (KARP, 1972), que listou 21 problemas fundamentais de otimização combinatória como NP-completos. Na formulação apresentada por Karp, o SCP é definido em sua versão de decisão: dado um conjunto universo U, uma coleção de subconjuntos $\mathscr{U}$ e um inteiro positivo k, deve-se determinar se existe um subconjunto $\mathscr{U}^* \subseteq \mathscr{U}$ com cardinalidade menor ou igual a k cuja união cubra todos os elementos de U. Essa definição consolidou o SCP como um dos pilares da teoria da complexidade, servindo de base para inúmeras formulações de otimização e aplicações práticas.

Em (GRAHAM; GRÖTSCHEL; LOVÁSZ, 1996), destaca-se que, para instâncias de tamanho moderado, a enumeração completa de todas as soluções possíveis torna-se computacionalmente inviável. Isso justifica a necessidade de algoritmos eficientes que explorem a estrutura do problema. Os autores apontam a Programação Linear (LP) como uma abordagem de sucesso, embora o desafio resida na modelagem de um poliedro frequentemente definido por um número exponencial de restrições. Quando a busca pela solução ótima exata é impraticável, o foco recai sobre heurísticas e meta-heurísticas capazes de encontrar soluções de alta qualidade em tempo computacional razoável.

A relevância prática do SCP é evidenciada por suas diversas aplicações e extensões. Em um trabalho pioneiro, (REVELLE; TOREGAS; FALKSON, 1976) expandiram o

modelo de cobertura de localização para cenários mais complexos, adaptando o problema para a cobertura de arcos em uma rede e tratando a demanda como contínua ao longo das vias, em vez de concentrada em pontos discretos. Além disso, os autores abordaram a localização de ambulâncias, considerando no tempo de resposta tanto o trajeto da base ao local da ocorrência quanto do local ao hospital. Essas extensões demonstram a flexibilidade do SCP na modelagem de problemas reais, onde a simples cobertura de pontos discretos pode ser insuficiente.

Dada a complexidade do SCP, diversas meta-heurísticas têm sido propostas na literatura. Em (CRAWFORD et al., 2014), por exemplo, apresenta-se uma aplicação do algoritmo de Colônia de Abelhas Artificiais (ABC), baseado em inteligência de enxame, para resolver o SCP com custos não unitários. O algoritmo emprega um mecanismo de busca que equilibra a exploração de novas regiões do espaço de soluções e a intensificação em torno de soluções promissoras. Os resultados experimentais indicaram que a abordagem ABC é competitiva, sendo capaz de encontrar a solução ótima em diversas instâncias.

No campo das abordagens evolutivas, os Algoritmos Genéticos (GAs) ocupam posição de destaque. Um trabalho relevante é o de (AICKELIN, 2002), que propõe um GA "indireto"para o SCP. Nessa abordagem, o algoritmo não otimiza a solução final diretamente; em vez disso, busca a melhor permutação dos elementos a serem cobertos. Um decodificador determinístico utiliza essa permutação para construir uma solução viável, a qual é posteriormente refinada por uma busca local para eliminar redundâncias. Essa separação de tarefas, onde o GA explora o espaço de ordenações e um decodificador constrói a solução, é uma técnica sólida para problemas combinatórios e alinha-se aos princípios utilizados no presente trabalho.

Recentemente, a variante abordada nesta pesquisa, o SCP com Conflitos (SCP-CS), foi introduzida por (CARRABS et al., 2024). Nesse problema, a seleção simultânea de pares de conjuntos conflitantes é permitida, porém incorre em uma penalidade somada à função objetivo. O intuito é minimizar o custo total, composto pelos custos dos conjuntos selecionados e pelas penalidades dos conflitos gerados. Os autores propuseram três formulações matemáticas para o SCP-CS e desenvolveram um algoritmo GRASP (*Greedy Randomized Adaptive Search Procedure*) paralelo, no qual as *threads* compartilham informações para evitar redundância de processamento e acelerar a convergência.

CAPÍTULO 3

HEURÍSTICA BRKGA

O *Biased Random-Key Genetic Algorithm* (BRKGA) é uma meta-heurística evolutiva proposta em (GONCALVES; RESENDE, 2010). Diferente dos algoritmos genéticos clássicos, que geralmente codificam a solução diretamente e requerem operadores de cruzamento e mutação específicos para cada problema, o BRKGA utiliza uma representação indireta e padronizada.

No BRKGA, cada indivíduo é representado por um cromossomo composto por um vetor de n chaves aleatórias (*random keys*), que são números reais gerados no intervalo $[0,1)$. O processo evolutivo ocorre exclusivamente nesse espaço contínuo (genótipo), de forma independente do problema. A conexão com o problema real é feita por meio de um *decodificador*, um algoritmo determinístico que traduz o vetor de chaves em uma solução viável (fenótipo) e calcula sua aptidão (*fitness*).

A dinâmica evolutiva do BRKGA baseia-se em três princípios fundamentais para gerar a próxima população (P_{k+1}) a partir da atual (P_k):

1. **Elitismo:** Uma porcentagem p_e dos melhores indivíduos (elite) é copiada integralmente para a próxima geração.

2. **Mutantes:** Uma porcentagem p_m da nova população é composta por vetores gerados totalmente de forma aleatória, introduzindo diversidade genética sem depender de perturbações em indivíduos existentes.

3. **Cruzamento Enviesado:** Os indivíduos restantes são gerados pelo cruzamento entre um pai selecionado aleatoriamente da elite e um pai não-elite. Um parâmetro $\rho > 0.5$ garante que o descendente herde, com maior probabilidade, os alelos do pai da elite.

O Algoritmo 1 apresenta a estrutura geral do funcionamento do BRKGA.

Algoritmo 1: Framework Geral do BRKGA

Entrada : Tam. cromossomo n; População p; Parâmetros p_e, p_m, ρ; Critério de parada.

Saída : Melhor solução encontrada.

1 Gerar pop. inicial P com p vetores aleatórios em $[0,1)^n$
2 Decodificar e avaliar cada indivíduo de P
3 **while** *critério de parada não atingido* **do**
4 Ordenar P por fitness e particionar em Elite (P_e) e Não-Elite ($P_{\bar{e}}$)
5 $P_m \leftarrow$ Gerar $p_m \times p$ novos vetores aleatórios `/* Mutantes */`
6 $P_c \leftarrow \emptyset$ `/* Cromossomos filhos */`
7 **while** $|P_e| + |P_m| + |P_c| < p$ **do**
8 Selecionar aleatoriamente pai $A \in P_e$ e pai $B \in P_{\bar{e}}$
9 Gerar $Filho$ onde cada gene $i \leftarrow \begin{cases} A[i], & \text{com probabilidade } \rho \\ B[i], & \text{caso contrário} \end{cases}$
10 $P_c \leftarrow P_c \cup \{Filho\}$
11 Decodificar e avaliar apenas os indivíduos em $P_m \cup P_c$
12 $P \leftarrow P_e \cup P_m \cup P_c$ `/* Atualiza a população para a próxima geração */`
13 **return** *Melhor solução em P*

Nas seções seguintes, detalha-se como esse framework geral foi adaptado para o SCP-CS, incluindo o decodificador híbrido e a utilização do Modelo de Ilhas para paralelismo e diversidade.

3.1 Pré-processamento: Pesos de Escassez

Uma etapa fundamental, realizada antes da execução do processo evolutivo, é o cálculo do peso de escassez w para cada elemento do conjunto universo. Essa métrica baseia-se na frequência de cobertura: quanto mais subconjuntos da instância $\mathscr{U}$ cobrem um

determinado elemento e, menor é seu peso. Inversamente, elementos cobertos por poucos subconjuntos ("raros") recebem um peso maior e são considerados prioritários. A Equação 1 apresenta o cálculo de w_e:

$$w_e = \frac{1}{|\{S_j \in \mathscr{U} : e \in S_j\}|} \tag{1}$$

O objetivo é guiar o decodificador: subconjuntos que cobrem muitos elementos de baixo peso são penalizados, pois a existência de múltiplas alternativas para cobri-los sugere que a escolha deste subconjunto específico pode gerar redundância. Por outro lado, subconjuntos que cobrem elementos de alto peso são priorizados, garantindo que itens de difícil cobertura sejam atendidos prontamente, minimizando o risco de inviabilidade nas etapas finais da construção da solução.

3.2 Decodificador Híbrido

O decodificador é o componente determinístico responsável por mapear o genótipo (cromossomo de chaves aleatórias) em um fenótipo (solução viável). Para exemplificar, considere um vetor de 4 alelos (representando subconjuntos U_1 a U_4) com os valores: $[0.5, 0.48, 0.64, 0.38]$. O valor da chave indica a prioridade: neste caso, U_3 possui a maior chave (0.64) e, portanto, a maior prioridade de inserção.

O processo inicia-se com a ordenação dos índices dos subconjuntos de forma decrescente, baseada em suas chaves. Para otimizar o desempenho computacional, adota-se uma estratégia de busca restrita: quando a solução parcial ainda possui baixa cobertura, analisa-se apenas os m subconjuntos de maior prioridade. Conforme a solução se aproxima da completude e torna-se difícil encontrar candidatos válidos, o espaço de busca é ampliado para incluir todos os subconjuntos restantes.

Uma iteração do decodificador consiste em percorrer a lista de prioridades e decidir pela inclusão de um subconjunto. Esse processo repete-se enquanto a solução for inviável (ou seja, enquanto houver elementos descobertos). Nas primeiras iterações, a cobertura é baixa, implicando em um grande número de candidatos válidos; analisar todos seria custoso. Assim, limita-se a busca. Por exemplo, pode-se definir que, enquanto a cobertura for menor que 90%, a busca restringe-se aos m melhores candidatos; caso contrário, a busca torna-se global.

Um subconjunto é considerado um "candidato válido"se, e somente se, sua inserção adicionar pelo menos um novo elemento à cobertura. Subconjuntos que contêm apenas elementos já cobertos são descartados para evitar redundância e custos desnecessários.

3.2.1 Métrica de Seleção

A decisão de incluir um candidato válido na solução é refinada por uma métrica de eficiência denominada *Quociente do Subconjunto* (q_j). Seja S a solução parcial atual e U_{nc} o conjunto de elementos ainda não cobertos. O quociente é definido por:

$$q_j = \frac{c_j + \text{PenalidadeAtual}(U_j)}{\sum_{e \in U_j \cap U_{nc}} w_e} \tag{2}$$

O numerador representa o custo marginal da inserção, somando o custo original do subconjunto (c_j) às penalidades geradas por conflitos com os subconjuntos já presentes em S. O denominador representa o benefício ponderado, calculado pela soma dos pesos dos novos elementos que U_j cobrirá.

O decodificador busca minimizar q_j. A cada iteração, seleciona-se o candidato que apresenta a melhor relação custo-benefício. Caso a fase de construção termine sem cobertura total (o que pode ocorrer devido à restrição do espaço de busca), aciona-se uma **Fase de Reparo**. Nesta etapa, percorre-se a lista de elementos descobertos e seleciona-se, para cada um, o subconjunto de menor custo efetivo que o contenha, assegurando a viabilidade da solução final.

3.2.2 Algoritmo do Decodificador

O Algoritmo 2 detalha o funcionamento do método híbrido, integrando a construção gulosa baseada em prioridades e a fase de reparo.

A entrada do decodificador é um cromossomo (um vetor de chaves aleatórias entre 0 e 1), as características da instância do SCP-CS e os pesos dos elementos. A saída é o custo da solução construída. Na primeira linha são inicializados como vazios o conjunto de elementos cobertos pela solução e o próprio conjunto solução. Na segunda linha a lista L é inicializada como uma lista dos subconjuntos em ordem decrescente de prioridade com base nos genes do cromossomo. Na linha 3 inicia-se o loop while que é executado enquanto a solução não cobre todos os elementos do conjunto universo. Nas linhas 4 e 5 são inicializados o Lim_{RCL}, que define o escopo da busca pelo melhor candidato, a variável s^* armazenará o melhor candidato encontrado e m^* armazenará a o quociente do melhor candidato. O loop for iniciado na linha 6 é o responsável por procurar o candidato de menor quociente no escopo definido, avaliando um subconjunto por vez. Calcula o custo total do subconjunto e seu quociente, armazenando-o em m_j. O $Beneficio(j)$ é a soma dos pesos dos elementos do subconjunto j.

A condicional na linha 9 verifica se o candidato analisado apresenta um quociente menor que o menor quociente encontrado até o momento, se sim, s^* é atualizado,

Algoritmo 2: Decodificador Híbrido (Construção Gulosa + Reparo)

Entrada : *Cromossomo*: Vetor de chaves aleatórias (genes).
I: Instância SCP-CS com M elementos.
Pesos: Vetor de pesos baseados na raridade.
Saída : Custo total da solução decodificada.

1 $Cob \leftarrow \emptyset; S \leftarrow \emptyset$ `/* Elementos cobertos e Conjunto Solução */`
2 $L \leftarrow$ Índices dos subconjuntos ordenados pelos genes (decrescente)
`/* FASE 1: Construção Gulosa Inteligente */`
3 **while** $|Cob| < M$ **do**
4 $\quad Lim_{RCL} \leftarrow$ calcularRangeBusca($|Cob|$)
5 $\quad s^* \leftarrow$ nulo; $m^* \leftarrow \infty$ `/* Melhor candidato e sua métrica */`
6 $\quad$ **for** $j \in L[1 \ldots Lim_{RCL}]$ *cujo* $Beneficio(j) > 0$ **do**
7 $\quad\quad custo' \leftarrow custo_j +$ penalidade_conflitos(j, S)
8 $\quad\quad m_j \leftarrow custo'/Beneficio(j)$
9 $\quad\quad$ **if** $m_j < m^*$ **then**
10 $\quad\quad\quad s^* \leftarrow j; m^* \leftarrow m_j$
11 $\quad$ **if** $s^* = nulo$ **then**
12 $\quad\quad$ **break** `/* Fim da Fase 1 (Fallback) */`
13 $\quad S \leftarrow S \cup \{s^*\}$
14 $\quad$ Atualizar Cob e penalidades de conflito
`/* FASE 2: Reparo de Emergência (Element-Oriented) */`
15 **for** *cada elemento* $e \in I$ *tal que* $e \notin Cob$ **do**
16 $\quad Candidatos \leftarrow I.ListaInvertida[e]$ `/* Subconjuntos que cobrem e */`
17 $\quad s^* \leftarrow \arg\min_{c \in Candidatos}\{custo_c +$ penalidade_conflitos$(c, S)\}$
18 $\quad S \leftarrow S \cup \{s^*\}$
19 $\quad$ Atualizar Cob e penalidades de conflito
20 **return** *CalcularCustoTotal(S, I)*

assim como m^*, ou seja, foi encontrado um novo melhor candidato. Após analisar todos os candidatos no escopo, o loop for iniciado na linha 6 é encerrado. Se nenhum candidato válido foi encontrado, o loop while é encerrado, caso contrário, o conjunto solução é atualizado com os novos elementos e acréscimo de custo.

A Fase 2 (linha 15) é ativada apenas se a solução ainda estiver incompleta. Itera-se sobre cada elemento e não coberto, identificando todos os subconjuntos que o contêm. Seleciona-se aquele com o menor custo efetivo para garantir a viabilidade. Por fim, retorna-se o custo total da solução.

3.3 Dinâmica Evolutiva Específica

Após a decodificação e avaliação dos indivíduos, aplica-se a dinâmica evolutiva do BRKGA para gerar a próxima população. O processo segue os princípios gerais descritos na introdução deste capítulo.

Em cada geração, a população é classificada pelo *fitness*. Define-se o conjunto Elite (P_e) e o conjunto Não-Elite ($P_{\bar{e}}$). A nova geração é formada pela cópia da elite, introdução de mutantes e cruzamento enviesado, conforme ilustrado na Figura 2.

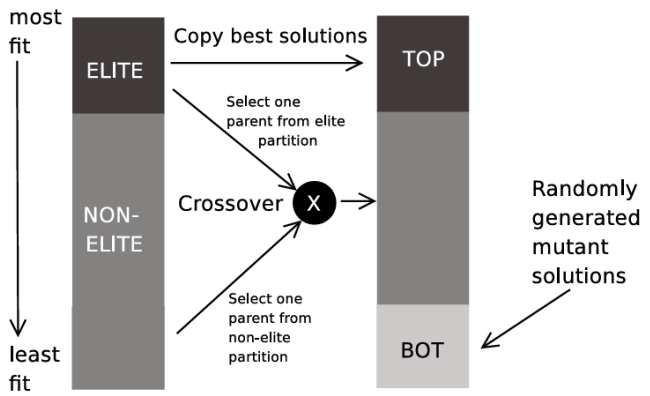

Figura 2 – Transição da população k para a população $k+1$.

O mecanismo de *crossover* (Figura 3) assegura a herança de características de alta qualidade. Para cada alelo, um sorteio probabilístico (baseado em $\rho > 0.5$) define se o gene será herdado do pai da elite ou do pai não-elite.

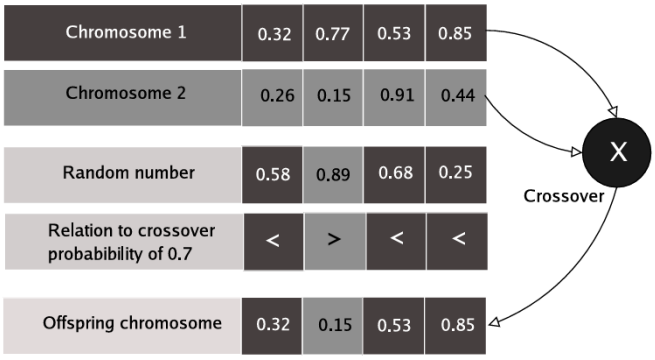

Figura 3 – Estratégia de cruzamento do BRKGA.

3.4 Modelo de Ilhas (BRKGA-Island-Model)

Para evitar a convergência prematura, este trabalho implementa o Modelo de Ilhas. Inicializam-se K ilhas, cada uma contendo uma população independente que evolui conforme os operadores descritos.

A comunicação entre as ilhas ocorre por meio de migrações periódicas. A cada G_{mig} gerações, os melhores indivíduos da ilha i são copiados e substituem os piores indivíduos da ilha vizinha (topologia em anel). Essa injeção de diversidade genética auxilia o algoritmo a escapar de ótimos locais. O Algoritmo 3 apresenta a visão geral do método proposto.

Algoritmo 3: BRKGA Multi-Ilhas para o SCP-CS

Entrada : I: Instância, P: População, K: Ilhas, T_{max}: Tempo limite.
Saída : Melhor solução global S^*.

1 $Pesos \leftarrow$ calcularPesosEscassez(I)
2 Inicializar e avaliar K ilhas independentes $(Pop_1, \ldots, Pop_K)$ em paralelo
3 $S^* \leftarrow$ Melhor indivíduo de todas as ilhas
4 **while** *tempo decorrido* $< T_{max}$ **do**
5 **for** *cada ilha* $k \in \{1 \ldots K\}$ **do**
6 $P_{elite} \leftarrow$ Selecionar as melhores soluções de Pop_k
7 $P_{mutantes} \leftarrow$ Gerar novos cromossomos aleatórios
 `/* Filhos gerados por pai Elite e pai Não-Elite (prob. ρ) */`
8 $P_{cruz} \leftarrow$ Cruzar P_{elite} e $(Pop_k \setminus P_{elite})$
9 $Pop_k \leftarrow P_{elite} \cup P_{mutantes} \cup P_{cruz}$
10 Avaliar $P_{mutantes} \cup P_{cruz}$ via Decodificador e atualizar S^*
11 **if** *critério de migração atingido* **then**
12 Substituir piores indivíduos pelas cópias dos melhores da ilha vizinha
13 **return** S^*

A entrada consiste na instância do problema, parâmetros populacionais e no tempo limite para a execução do algoritmo. Inicialmente, são calculados os pesos dos elementos, as K ilhas são inicializadas com populações aleatórias e aos indivíduos é aplicada a função *fitness*. A variável S^* é inicializada como o indivíduo de menor custo entre todas as ilhas. O loop while é iniciado e permanece em execução até que o critério de parada seja atingido. Ele é o responsável por gerar uma nova população para cada ilha. A

nova população da ilha é composta pelos indivíduos de elite da anterior, os mutantes e os indivíduos resultantes dos cruzamentos.

O decodificador é utilizado para avaliar cada um desses indivíduos e S^* é atualizado caso seja encontrada uma nova melhor solução. Se o critério de migração for atingido, os piores indivíduos de uma ilha são substituídos pelos melhores da ilha vizinha.

Esse processo de gerar novas populações ocorre para cada uma das ilhas até que o critério de parada seja atingido, em seguida, o melhor indivíduo encontrado entre todas as ilhas é retornado e o algoritmo encerrado.

CAPÍTULO 4

RESULTADOS COMPUTACIONAIS

Este capítulo apresenta e discute os resultados experimentais obtidos com a aplicação do algoritmo BRKGA-Island-Model proposto. A implementação foi desenvolvida em linguagem C++, utilizando a *Standard Template Library* (STL) para otimização de estruturas de dados e gerenciamento eficiente de memória.

Para a validação da abordagem, utilizou-se o conjunto de instâncias de referência (*benchmark*) proposto na literatura por (CARRABS et al., 2024). Essa escolha permite a comparação direta da qualidade das soluções e do tempo computacional com o estado da arte. As instâncias estão disponíveis publicamente em repositório online[1].

4.1 Caracterização das Instâncias e Ambiente de Teste

O conjunto de testes é composto por 80 instâncias divididas em grupos com características variadas de densidade e dimensão. A Tabela 1 detalha esses grupos: a coluna **Grupo** identifica a família da instância; **Elementos** ($|U|$) indica o tamanho do universo a ser coberto; e **Subconjuntos** ($|\mathscr{U}|$) representa a quantidade de subconjuntos disponíveis para escolha.

[1] <https://github.com/lmores/or-scp-cs-src>

Tabela 1 – Caracterização das instâncias de teste utilizadas (*Benchmark* SCP-CS).

Grupo	**Instâncias**	**Elementos** ($\|U\|$)	**Subconjuntos** ($\|\mathscr{U}\|$)
4	scp41-3, ..., scp410-3	200	334
5	scp51-3, ..., scp510-3	200	667
6	scp61-3, ..., scp65-3	200	334
a	scpa1-3, ..., scpa5-3	300	1000
b	scpb1-3, ..., scpb5-3	300	1000
c	scpc1-3, ..., scpc5-3	400	1334
d	scpd1-3, ..., scpd5-3	400	1334
e	scpe1-3, ..., scpe5-3	50	167
clr10	scpclr10-3	511	70
clr11	scpclr11-3	1023	110
clr12	scpclr12-3	2047	165
clr13	scpclr13-3	4095	239
cyc06	scpcyc06-3	240	64
cyc07	scpcyc07-3	672	150
cyc08	scpcyc08-3	1792	342
cyc09	scpcyc09-3	4608	768
cyc10	scpcyc10-3	11520	1707
nre	scpnre1-3, ..., scpnre5-3	500	1667
nrf	scpnrf1-3, ..., scpnrf5-3	500	1667
nrg	scpnrg1-3, ..., scpnrg5-3	1000	3334
nrh	scpnrh1-3, ..., scpnrh5-3	1000	3334

4.2 Parametrização do Algoritmo

A Tabela 2 apresenta a configuração dos parâmetros do BRKGA-Island-Model. Os valores foram determinados empiricamente através de testes preliminares, buscando um equilíbrio entre diversificação (exploração do espaço de busca) e intensificação (refinamento da solução).

Conforme descrito na Tabela 2 , em cada instância, o tamanho total da população é o número de subconjuntos multiplicado por 1.5 e o valor resultante é dividido entre as 4 ilhas, definindo assim o número de indivíduos por ilha. Foi estabelecido o tamanho máximo da população como 1800 indivíduos por questões de performance. Os critérios de parada são de tempo, 600 segundos no máximo, e 150 gerações, aquele que for alcançado primeiro interrompe a execução do algoritmo. A frequência de migração (ou G_{mig}) representa o número de gerações que se passam até que ocorram migrações de indivíduos entre as ilhas e o número de migrantes é a quantidade de indivíduos que migrarão em cada ilha. A tolerância de conflito é a quantidade máxima de elementos que cada par de subconjuntos de uma mesma instância pode ter em comum sem que entrem em conflito, ou seja, sem que haja penalidade por acrescentar ambos ao conjunto solução.

Tabela 2 – Parâmetros configurados para os experimentos computacionais.

Parâmetro	**Valor**
Tamanho da População Total	$1.5 \times \lvert\mathscr{U}\rvert$
Número de Ilhas (K)	4
Tempo Máximo de Execução	600 segundos
Critério de Parada (Gerações)	150 gerações
Frequência de Migração (G_{mig})	A cada 20 gerações
Taxa de Migração	10 indivíduos (melhores/piores)
Tolerância de Conflito (k)	1 elemento
Probabilidade de Herança da Elite (ρ)	0.6
Percentual de Mutantes	25% da população da ilha
Percentual de Elite	15% da população da ilha
LRC - Subconjuntos Prioritários (m)	40% de $\lvert\mathscr{U}\rvert$

O valor ρ é o responsável pela característica enviesada do BRKGA, ele representa a chance de, em um *crossover*, o novo indivíduo receber um gene do pai de elite em detrimento de um gene do pai selecionado da população não-elite. O percentual de mutantes e de elite é definido a partir do tamanho da população da ilha, a cada geração, para cada ilha, é introduzido um número de indivíduos gerados aleatoriamente, valor definido como 25% da população da ilha e os 15% melhores indivíduos da ilha são considerados como elite. Por fim, o parâmetro que indica o percentual de subconjuntos prioritários (m) representa quantos subconjuntos são analisados pelo decodificador no momento da construção da solução em que a cobertura da mesma é menor que 90% dos elementos. Esse parâmetro é importante pois o decodificador analisa somente os m de maior prioridade ao invés de analisar todos os subconjuntos possíveis, o que economiza tempo nas iterações iniciais. A partir do momento em que a solução já cobre mais de 90% dos elementos, todos os subconjuntos são analisados, a área de busca é ampliada ao final pois quanto maior a cobertura mais difícil é encontrar candidatos válidos.

Os resultados consolidados dos experimentos computacionais são apresentados na Tabela 3. Para facilitar a análise, os dados foram agrupados conforme as classes de instâncias definidas na literatura (por exemplo, o grupo `scp4` representa a média aritmética das instâncias `scp41-3` a `scp410-3`).

Os experimentos computacionais foram conduzidos em um computador equipado com processador Intel Core i5-11400H (2.70 GHz), com 6 núcleos físicos e 12 lógicos, 16 GB de memória RAM.

A tabela reporta três métricas fundamentais de desempenho para a comparação entre o Parallel GRASP e o BRKGA-Island proposto:

- $\bar{t}_{melhor}$ **(s):** Tempo médio (em segundos) decorrido até o algoritmo encontrar sua melhor solução;
- $\bar{t}_{total}$ **(s):** Tempo médio total de execução até o encerramento do algoritmo;
- $\overline{\textbf{Gap}}$ **(%):** Diferença percentual média de custo entre os métodos. Valores negativos indicam que o BRKGA-Island obteve, em média, soluções de melhor qualidade (menor custo) que o método comparativo naquele grupo.

As tabelas contendo os resultados detalhados por instância individual encontram-se no Apêndice A.

Tabela 3 – Resultados médios agrupados por conjunto e Média Geral.

Grupo	**Parallel GRASP (Médias)**		**BRKGA-Island (Médias)**		
	$\bar{t}_{melhor}$ **(s)**	$\bar{t}_{total}$ **(s)**	$\bar{t}_{melhor}$ **(s)**	$\bar{t}_{total}$ **(s)**	$\overline{\textbf{Gap}}$ **(%)**
scp4	76.21	128.82	6.11	37.49	0.12%
scp5	146.46	236.78	75.06	159.91	0.75%
scp6	32.64	61.71	6.67	37.01	-0.70%
scpa	348.15	516.61	194.59	579.25	-3.84%
scpb	141.49	257.13	222.91	570.62	-6.50%
scpc	363.69	590.39	482.98	603.57	-5.71%
scpd	111.30	377.41	288.36	603.55	-8.79%
scpe	0.44	7.62	0.85	3.05	5.52%
scpclr	18.00	42.19	13.54	195.99	0.00%
scpcyc	173.43	371.78	269.69	278.05	9.38%
scpnre	291.72	562.12	70.80	609.34	-4.92%
scpnrf	27.05	568.28	55.11	609.97	0.52%
scpnrg	96.74	611.29	396.75	616.11	-6.17%
scpnrh	67.54	616.85	363.23	635.62	-10.08%
Média Geral	**135.35**	**353.50**	**174.76**	**395.68**	**-2.17%**

Dados do Parallel GRASP extraídos de (CARRABS et al., 2024).

A análise dos dados apresentados na Tabela 3 revela que o algoritmo BRKGA-Island atingiu um desempenho altamente competitivo em relação ao estado da arte, as médias de *Gap* obtidas foram negativas ou nulas na maioria dos grupos testados. O método demonstrou robustez especialmente nas classes de instâncias de maior dimensão, superando o algoritmo Parallel GRASP em cenários complexos. Nos grupos **scpnrh** e **scpd**, por

exemplo, foram obtidas reduções médias de custo de $10,08\%$ e $8,79\%$, respectivamente. Esse comportamento sugere que a estratégia evolutiva combinada ao modelo de ilhas, é eficaz na exploração de vastos espaços de busca, permitindo que o algoritmo escape de ótimos locais onde métodos construtivos tradicionais tendem a estagnar. Casos individuais reforçam essa observação, como na instância `scpnrh4-3` (detalhada na Tabela 4 do Apêndice), onde a redução de custo ultrapassou a marca de 15%.

Além da qualidade das soluções, o BRKGA-Island apresentou rápida convergência, particularmente em instâncias de pequeno e médio porte. Nos grupos `scp4`, `scp6` e `scpnre`, a média de tempo até a melhor solução ($\bar{t}_{melhor}$) foi consideravelmente inferior à do método comparativo. Entretanto, limitações foram observadas em cenários específicos, como no grupo **scpe**, que apresentou um *Gap* médio positivo de $5,52\%$, indicando uma leve superioridade do GRASP em instâncias pequenas e esparsas.

A degradação de desempenho mais expressiva, contudo, concentrou-se no grupo **scpcyc**, que registrou um *Gap* médio de $9,38\%$. Uma análise aprofundada da estrutura dessas instâncias revela que esse comportamento decorre de uma característica comum a todas as instâncias do grupo: todos os subconjuntos possuem custo idêntico e cobrem uniformemente 3 ou 4 elementos. Consequentemente, a métrica gulosa utilizada pelo decodificador, baseada na razão entre custo e benefício de cobertura, perde seu poder discriminatório, tornando-se incapaz de diferenciar a qualidade dos candidatos. Sem um gradiente claro, o processo de seleção aproxima-se de uma escolha aleatória, fenômeno que se torna ainda mais claro com o aumento da dimensão do problema. A instância `scpcyc10-3`, com mais de 11 mil elementos, exemplifica esse colapso da heurística, atingindo um custo $37,45\%$ superior ao método de referência.

Apesar das dificuldades pontuais impostas pela estrutura do grupo cíclico, a análise global valida a eficácia da abordagem proposta. Considerando a totalidade dos experimentos, o BRKGA-Island foi capaz de entregar uma solução de custo igual ou inferior à do Parallel GRASP em 66 das 79 instâncias testadas. Esse resultado consolida o método como uma alternativa robusta e eficiente para a resolução do SCP-CS, capaz de aliar qualidade de solução e tempos computacionais competitivos na grande maioria dos cenários práticos.

CAPÍTULO 5

CONCLUSÃO

O presente trabalho dedicou-se ao estudo e à resolução do Problema de Cobertura de Conjuntos com Conflitos (SCP-CS), um problema de otimização combinatória de alta complexidade computacional e notável relevância prática. O objetivo central consistiu no desenvolvimento de uma abordagem baseada em meta-heurísticas evolutivas capaz de competir com o estado da arte. Para tanto, projetou-se e implementou-se o algoritmo BRKGA-Island, utilizando a linguagem C++ para maximizar a eficiência no gerenciamento de memória e processamento.

A metodologia proposta integrou a robustez do BRKGA padrão a uma estratégia de migração entre ilhas independentes, visando preservar a diversidade genética e reduzir a convergência prematura para ótimos locais. Um elemento crucial para o sucesso da abordagem foi o desenvolvimento de um decodificador híbrido, combinando uma fase construtiva gulosa e uma fase de reparo projetada para garantir a viabilidade das soluções geradas.

A análise dos experimentos computacionais, realizados sobre instâncias retiradas da literatura, evidenciou a competitividade do BRKGA-Island frente ao algoritmo Parallel GRASP. O método proposto superou a referência em uma ampla variedade de cenários, demonstrando capacidade de obter soluções de custo inferior e convergindo frequentemente para resultados de alta qualidade com eficiência temporal superior. A

predominância de resultados positivos valida a eficácia da arquitetura de ilhas e do decodificador desenvolvidos, demonstrando a robustez da estratégia como uma alternativa de qualidade para a resolução de problemas complexos de cobertura.

O estudo abre perspectivas para investigações futuras visando aprimorar o desempenho do algoritmo e expandir sua aplicabilidade. Uma direção natural para a continuidade desta pesquisa é a hibridização com métodos de busca local, como *Hill Climbing* ou *Variable Neighborhood Descent* (VND), aplicados após a decodificação para refinar as soluções. Adicionalmente, considerando a sensibilidade do método a estruturas uniformes (como observado no grupo **scpcyc**), sugere-se o desenvolvimento de decodificadores adaptativos ou estratégias de *path-relinking* para diversificar a busca. Por fim, a investigação de abordagens metaheurísticas, integrando o componente evolutivo a solvers exatos para resolver subproblemas fixados, apresenta-se como um caminho promissor para elevar a qualidade das soluções em instâncias de alta dimensão.

APÊNDICE A

RESULTADOS DETALHADOS POR INSTÂNCIA

Apresentamos neste apêndice os resultados detalhados obtidos para cada uma das instâncias de teste, comparando o desempenho do Parallel GRASP com o BRKGA-Island. Os dados do Parallel GRASP estão disponíveis em (CARRABS et al., 2024) e o BRKGA-Island foi executado 10 vezes para cada instância e os resultados apresentados referem-se aos dados da execução na qual foi obtido o menor custo para a respectiva instância. A Tabela 4 demonstra os resultados obtidos da execução de 79 instâncias do SCP-CS.

Tabela 4 – Comparação de resultados: Parallel GRASP vs. BRKGA-Island (Proposto).

Instância	**Parallel GRASP**			**BRKGA-Island**			
	Melhor	$\mathbf{t}_{melhor}$ **(s)**	$\mathbf{t}_{total}$ **(s)**	**Melhor**	$\mathbf{t}_{melhor}$ **(s)**	$\mathbf{t}_{total}$ **(s)**	**Gap (%)**
scp41-3	2097	68,83	110,11	2097	6,19	39,32	0,00%
scp42-3	1977	66,77	105,20	1977	4,51	40,63	0,00%
scp43-3	2583	87,81	134,53	2583	6,48	38,02	0,00%
scp44-3	2543	166,52	209,64	2575	5,59	38,43	1,24%
scp45-3	2247	74,94	111,58	2247	2,65	35,36	0,00%
scp46-3	2602	91,50	133,59	2602	4,63	36,36	0,00%
scp47-3	2128	2,00	96,92	2128	4,35	36,48	0,00%
scp48-3	2663	84,67	129,61	2663	8,67	37,63	0,00%
scp49-3	2604	16,14	109,52	2604	9,34	37,15	0,00%
scp410-3	2501	102,95	147,56	2501	8,70	35,53	0,00%
scp51-3	1532	199,44	278,38	1532	43,43	160,19	0,00%
scp52-3	1446	30,66	189,63	1532	58,49	159,59	5,61%
scp53-3	1479	152,00	235,36	1479	47,81	156,22	0,00%
scp54-3	1360	89,91	263,18	1420	146,17	165,76	4,23%
scp55-3	1382	143,06	217,92	1382	27,49	172,61	0,00%
scp56-3	1410	172,99	247,16	1410	20,95	152,39	0,00%
scp57-3	1571	200,74	276,17	1633	19,67	168,43	3,80%
scp58-3	1517	129,86	197,42	1517	96,54	157,79	0,00%
scp59-3	1557	216,12	285,26	1574	143,92	154,03	1,08%
scp510-3	1435	95,67	182,38	1338	146,16	152,09	-7,25%
scp61-3	2930	0,70	53,08	2930	0,23	36,66	0,00%
scp62-3	3361	37,45	60,38	3327	2,76	33,07	-1,02%
scp63-3	4105	39,95	61,78	4085	15,54	36,79	-0,49%
scp64-3	3211	30,31	55,11	3211	0,26	38,53	0,00%
scp65-3	3181	54,81	78,20	3119	14,57	40,00	-1,99%
scpa1-3	2893	45,27	389,85	2824	165,18	560,92	-2,44%
scpa2-3	2934	338,52	489,32	2784	212,01	577,84	-5,39%
scpa3-3	2647	361,96	514,08	2546	126,32	572,78	-3,97%
scpa4-3	2701	539,39	601,24	2633	206,50	589,58	-2,58%
scpa5-3	3069	455,65	588,89	2928	262,93	595,16	-4,82%
scpb1-3	4320	150,86	255,21	3749	7,81	558,56	-15,23%
scpb2-3	4818	198,02	287,22	4567	447,68	567,57	-5,32%

Tabela 4 – Comparação de resultados: Parallel GRASP vs. BRKGA-Island (Proposto)

Instância	**Parallel GRASP**			**BRKGA-Island (Proposto)**			
	Melhor	$\mathbf{t}_{melhor}$ **(s)**	$\mathbf{t}_{total}$ **(s)**	**Melhor**	$\mathbf{t}_{melhor}$ **(s)**	$\mathbf{t}_{total}$ **(s)**	**Gap (%)**
scpb3-3	4099	159,67	251,77	3790	92,29	570,01	-8,15%
scpb4-3	3885	42,98	233,85	3610	154,45	577,19	-7,62%
scpb5-3	4542	156,44	257,60	4512	406,31	579,81	-0,66%
scpc1-3	7437	525,47	601,88	6945	564,47	601,26	-7,08%
scpc2-3	3995	88,92	580,51	3697	319,67	606,07	-8,06%
scpc3-3	4611	453,91	601,57	4358	525,98	607,42	-5,81%
scpc4-3	4457	394,38	566,19	4276	566,34	600,85	-4,23%
scpc5-3	5329	355,77	601,83	4929	438,44	602,29	-6,29%
scpd1-3	5340	109,91	407,39	4861	198,45	604,87	-9,85%
scpd2-3	5520	263,38	391,35	5233	331,50	603,00	-5,48%
scpd3-3	5572	29,03	333,22	5274	249,11	600,39	-5,65%
scpd4-3	7873	81,22	378,18	7375	411,43	604,55	-6,75%
scpd5-3	5718	72,97	376,96	5092	349,01	605,30	-12,29%
scpe1-3	26	1,52	9,25	29	0,33	2,93	10,34%
scpe2-3	28	0,09	7,39	28	2,14	3,24	0,00%
scpe3-3	24	0,05	5,89	24	0,04	3,09	0,00%
scpe4-3	25	0,12	8,39	29	1,63	3,26	13,79%
scpe5-3	28	0,44	7,19	29	0,12	2,78	3,45%
scpclr10-3	1926	1,03	21,69	1926	4,76	5,32	0,00%
scpclr11-3	3501	1,03	27,39	3501	1,41	27,84	0,00%
scpclr12-3	6429	21,31	40,45	6429	42,62	145,81	0,00%
scpclr13-3	13267	48,64	79,23	13267	5,35	605,00	0,00%
scpcyc06-3	126	0,47	13,80	126	0,16	1,32	0,00%
scpcyc07-3	335	1,23	41,36	335	4,56	13,53	0,00%
scpcyc08-3	908	6,83	599,41	916	132,63	164,34	0,87%
scpcyc09-3	2317	402,47	602,13	2517	602,58	602,58	7,95%
scpcyc10-3	5728	456,13	602,21	9158	608,51	608,51	37,45%
scpnre1-3	3711	333,32	581,88	3648	55,09	603,11	-1,73%
scpnre2-3	3693	263,22	495,97	3557	149,79	602,00	-3,82%
scpnre3-3	3826	137,64	591,86	3347	65,80	615,83	-14,31%
scpnre4-3	3726	157,58	574,05	3570	67,14	611,25	-4,37%
scpnre5-3	3660	297,39	566,87	3646	16,17	614,72	-0,38%
scpnrf1-3	1185	19,02	559,89	1185	21,09	618,93	0,00%

Tabela 4 – Comparação de resultados: Parallel GRASP vs. BRKGA-Island (Proposto)

Instância	**Parallel GRASP**			**BRKGA-Island (Proposto)**			
	Melhor	$\mathbf{t}_{melhor}$ **(s)**	$\mathbf{t}_{total}$ **(s)**	**Melhor**	$\mathbf{t}_{melhor}$ **(s)**	$\mathbf{t}_{total}$ **(s)**	**Gap (%)**
scpnrf2-3	1243	34,19	570,86	1254	99,39	608,93	0,88%
scpnrf3-3	1184	1,83	561,74	1205	24,05	609,11	1,74%
scpnrf4-3	1183	2,20	560,21	1183	42,64	605,12	0,00%
scpnrf5-3	1226	78,03	588,70	1226	23,29	607,76	0,00%
scpnrg1-3	13234	240,19	609,41	12675	612,19	612,19	-4,95%
scpnrg2-3	13110	48,30	607,63	12480	427,33	630,05	-5,05%
scpnrg3-3	14287	65,88	615,52	13695	270,20	617,27	-4,32%
scpnrg4-3	13673	38,61	613,43	12289	310,51	614,98	-11,26%
scpnrg5-3	13529	90,75	610,50	12851	363,52	605,50	-5,28%
scpnrh1-3	9315	129,91	616,55	8718	385,01	673,06	-6,85%
scpnrh2-3	9101	14,45	613,57	8625	82,45	643,32	-5,52%
scpnrh3-3	9294	12,13	617,05	8472	189,39	648,09	-9,70%
scpnrh4-3	9644	100,91	616,32	8331	638,53	638,53	-15,76%
scpnrh5-3	9461	80,34	620,76	8406	520,78	625,14	-12,55%

Dados do Parallel GRASP extraídos de (CARRABS et al., 2024).

REFERÊNCIAS

AICKELIN, U. An indirect genetic algorithm for set covering problems. *The Journal of the Operational Research Society*, Palgrave Macmillan Journals, v. 53, n. 10, p. 1118–1126, 2002. ISSN 01605682, 14769360. Disponível em: <http://www.jstor.org/stable/822971>. 6

CAPRARA, A.; TOTH, P.; FISCHETTI, M. Algorithms for the set covering problem. *Annals of Operations Research*, v. 98, p. 353–, 12 2000. 1

CARRABS, F. et al. Solving the set covering problem with conflicts on sets: A new parallel grasp. *Computers & Operations Research*, v. 166, 2024. ISSN 0305-0548. 3, 6, 15, 18, 22, 25

CRAWFORD, B. et al. Application of the artificial bee colony algorithm for solving the set covering problem. *The Scientific World Journal*, v. 2014, n. 1, p. 189164, 2014. Disponível em: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2014/189164>. 6

GONCALVES, J. F.; RESENDE, M. G. C. Biased random-key genetic algorithms for combinatorial optimization. *Journal Of Heuristics*, v. 17, p. 487–525, 2010. 7

GRAHAM, R. L.; GRÖTSCHEL, M.; LOVÁSZ, L. (Ed.). *Handbook of combinatorics (vol. 1)*. Cambridge, MA, USA: MIT Press, 1996. ISBN 0262071703. 5

KARP, R. M. Reducibility among combinatorial problems. In: MILLER, R. E.; THATCHER, J. W. (Ed.). *Proceedings of a symposium on the Complexity of Computer Computations*. [S.l.: s.n.], 1972. (The IBM Research Symposia Series), p. 85–103. 5

REVELLE, C.; TOREGAS, C.; FALKSON, L. Applications of the location set-covering problem. *Geographical Analysis*, v. 8, n. 1, p. 65–76, 1976. Disponível em: <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1538-4632.1976.tb00529.x>. 5